

atabase Systems and Laboratory

Entity-relationship model

Nicola Saccomanno

`nicola.saccomanno@uniud.it`

Bachelor degrees in Internet of things, Big Data, Machine Learning, & Computer Science
DMIF, University of Udine

A.Y. 2025/2026

Database design

The process of designing a database is typically articulated into different phases, that separate *what* to represent from *how* to represent it:

1. **Conceptual design**
2. Logical design
3. Physical design

Conceptual design

- ▶ The initial phase of database design is to **characterize fully the data needs** of the **prospective database users**
- ▶ Such needs are **encoded into a conceptual schema**, that acts as a connecting point between stakeholders and IT personnel
 - ▶ It encodes the **informal specification about the reality in a formal and complete description**
 - ▶ The core idea is to describe, at a **high abstraction level**, how data are organized
- ▶ Bare in mind the difference between specifications about data and those about operations:
 - ▶ Data: about the content of the DB
 - ▶ Operation: about the usage of the DB from users/applications
 - ▶ Conceptual design is about the former

Conceptual design

- ▶ In this course, for the development of the conceptual schema, we will consider the **Entity-Relationship** model, which makes use of *diagrams* to represent the overall logical structure of a database graphically
- ▶ Such diagrams are typically **complemented by free text annotations**, that describe the requirements that could not be encoded in the diagram

Entity-Relationship model

- ▶ The Entity-Relationship model describes a domain by means of:
 - ▶ **Entity sets**
 - ▶ **Relationship sets**
 - ▶ **Attributes**
- ▶ There are several ways in which entity sets, relationship sets and attributes can be represented in E-R diagrams
- ▶ We will consider the notation proposed in the book:

Database Systems - Concepts, Languages and Architectures

P. Atzeni, S. Ceri, S. Paraboschi, and R. Torlone

Entity sets

- ▶ An **entity** is an object that exists in the considered domain and is distinguishable from other objects
 - ▶ Example in the University domain: a specific professor, student, or course
- ▶ An **entity set** is a set of entities of the same type that share the same properties
 - ▶ Example: the set of all professors, each characterized by a name, a surname, and a salary
- ▶ An entity exists besides the property we decide to associate with it

Entity sets

Attributes

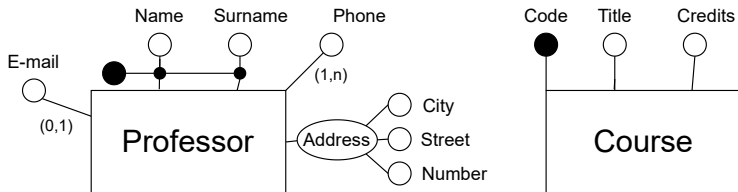
- ▶ The properties, possessed by all members of an entity set, are represented by **attributes**
- ▶ Each attribute can take on values of a given domain (i.e., a set of admissible values for the attribute)
 - ▶ Domains are typically not reported in the conceptual schema, but rather in the complementary documentation
 - ▶ Example: “Name and surname are string of variable lengths, of at most 50 characters”
- ▶ A subset of the attributes forms a **key** of the entity set, i.e, knowing the values of such attributes
 - ▶ it is possible to uniquely identify a member (entity) of the entity set
 - ▶ or, in other words there cannot be two entities in the entity set which have the same values over the key attributes

Entity sets

Notation

Entity sets

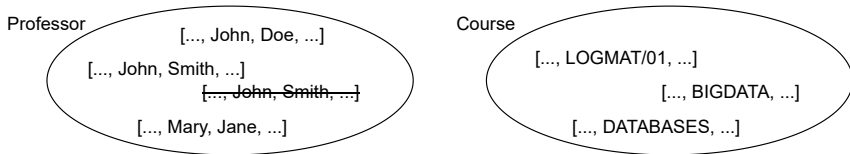
Notation



- ▶ Entity set *Professor* has the attributes:
 - ▶ *E-mail*, optional
 - ▶ *Name* and *Surname*, that together make the key
 - ▶ *Phone*, multi-valued
 - ▶ *Address*, composite, and made of *City*, *Street*, *Number*
- ▶ Entity set *Course* has the attributes:
 - ▶ *Code*, that makes the key
 - ▶ *Title*
 - ▶ *Credits*

Entity sets

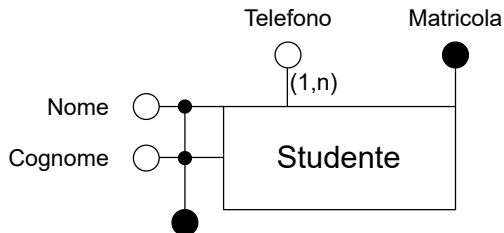
Intuitive meaning



- ▶ An entity set cannot contain two entities with the same values on the key attributes
 - ▶ Here: name and surname for Professor, code for Course
- ▶ Observe that, being a set, each entity set has a *trivial* key composed of all the attributes
- ▶ Finally, note that, in general, an entity may have more than one key (it really depends on the considered domain)

Entity sets

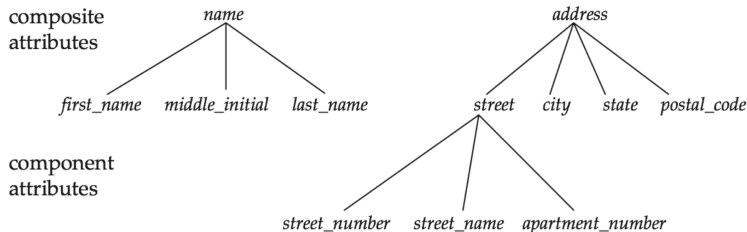
Multiple keys



- ▶ Entity set Student has two possible keys
- ▶ Spoiler: one key will be chosen as the primary key in the relational schema
- ▶ Thus, in our domain, we assume that *name* and *surname* are enough to discriminate between students (no homonyms are possible)

Other notes on the attributes

- ▶ Composite attributes can be nested further



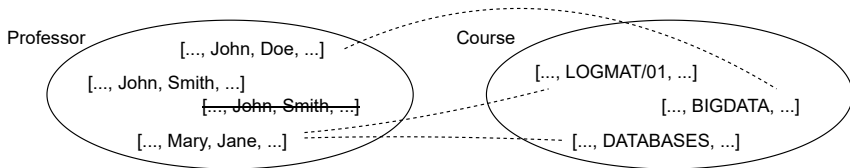
- ▶ The characteristics of attributes can be combined, e.g., there can be multi-valued composite attributes, as well as optional composite attributes

Relationship sets

- ▶ A **relationship** is an association among several entities
 - ▶ E.g., *Professor: [John, Doe]* may be linked with *Course: [BIGDATA]* by means of a relationship *Teaches*
 - ▶ The relationship is an association between actual entities
- ▶ A **relationship set** $R \subseteq E_1 \times E_2 \times \dots \times E_n$ is a mathematical relation among $n \geq 2$ entities (e_i), each taken from its entity set (E_i)
 - ▶ $R = \{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$
 - ▶ where $(e_1, e_2, \dots, e_n)$ is a *relationship*
 - ▶ note that we cannot have repeated entries (being R a set)
- ▶ For instance, $([John, Doe], [BIGDATA]) \in Teaches$ is a relationship belonging to relationship set *Teaches*

Relationship sets

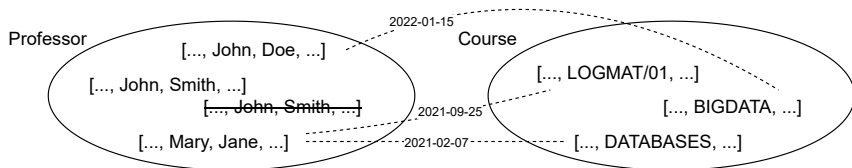
Intuitive meaning



- ▶ The picture shows three relationships (dashed lines), which we assume belonging to relationship set *Teaches*:
 - ▶ *John Doe* is associated to *BIGDATA*, (*[John, Doe]*, *[BIGDATA]*)
 - ▶ *Mary Jane* is associated to *LOGMAT/01*, (*[Mary, Jane]*, *[LOGMAT/01]*)
 - ▶ *Mary Jane* is associated to *DATABASES*, (*[Mary, Jane]*, *[DATABASES]*)

Relationships and attributes

- ▶ Attributes may also be associated with relationships, e.g., to track when a professor started teaching a course
- ▶ Intuitively, a relationship thus becomes as follows:
 - ▶ ([John, Doe], [BIGDATA], 2022 – 01 – 15)
- ▶ Still, the relationship is “defined” by the two entities that are linked, other attributes are “accessory” information. **Cannot have both:**
 - ▶ ([John, Doe], [BIGDATA], 2022 – 01 – 15)
 - ▶ ([John, Doe], [BIGDATA], 2023 – 01 – 20)



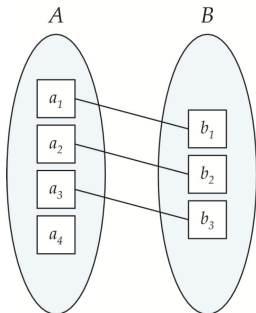
Relationship sets

Cardinality constraints

- ▶ Given entity sets E_1, E_2 , and a relationship set R between them, entities may participate in relationships in different ways, for instance:
 - ▶ An entity of E_1 may not participate in any relationship with an entity of E_2 (a prof. may not be teaching any courses)
 - ▶ An entity of E_1 may participate in > 1 relationships with an entity of E_2 (a prof. may be teaching more than one course)
- ▶ In general, for a binary relationship set we can identify the following **cardinalities**:
 - ▶ One to one
 - ▶ One to many
 - ▶ Many to one
 - ▶ Many to Many
- ▶ Also, entity sets may have a *partial* or *total* **participation** to a relationship set

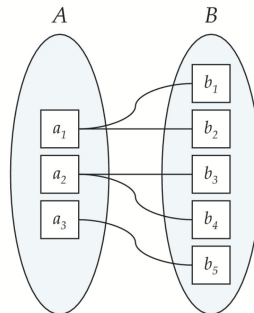
Relationship sets

One to one and one to many



(a)

One to one



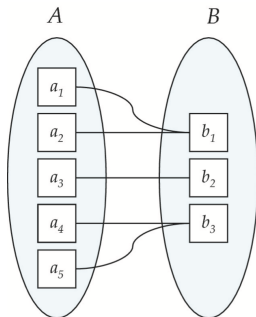
(b)

One to many

- ▶ Left: A has a partial participation, while B total
- ▶ Right: both A and B have a total participation

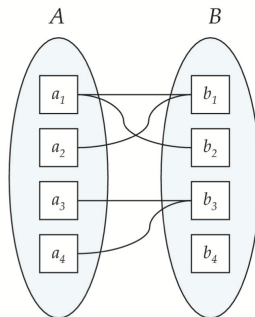
Relationship sets

Many to one and many to many



(a)

Many to
one



(b)

Many to many

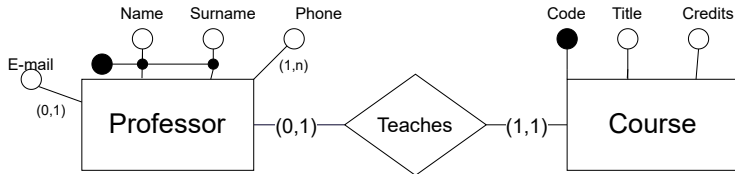
- ▶ Left: both A and B have a total participation
- ▶ Right: A has a total and B has a partial participation

Relationship sets

Notation for participation and cardinality constraints

Relationship sets

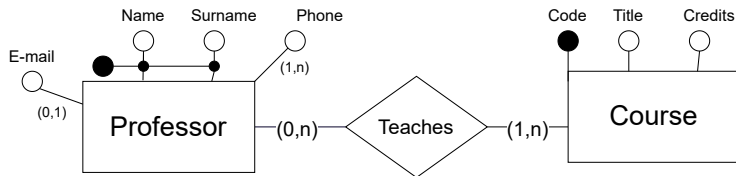
Notation for participation and cardinality constraints



- ▶ Cardinality constraints determine the number of times (min or max) an entity can occur in the relationship set
- ▶ A professor teaches in at most one course: $(0,1)$
- ▶ A course is taught by one and only one professor: $(1,1)$
- ▶ $(1,1)$ is assumed by default, and can be omitted
- ▶ Concerning the relationship set *Teaches*, this means that:
 - ▶ A given entity $p_1 \in \text{Professor}$ can appear in at most one relationship $(p_1, c) \in \text{Teaches}$ where $c \in C$
 - ▶ A given entity $c_1 \in \text{Course}$ appears in exactly one relationship $(p, c_1) \in \text{Teaches}$ where $p \in \text{Professor}$

Relationship sets

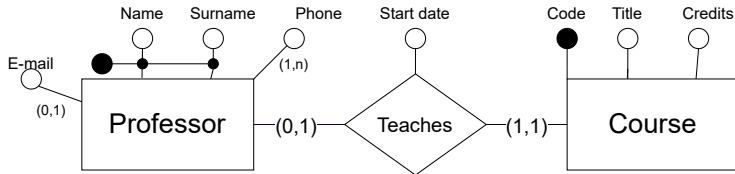
Notation for participation and cardinality constraints



- ▶ A professor teaches in zero or more courses: $(0, n)$
- ▶ A course is taught by one or more professors: $(1, n)$
- ▶ Concerning the relationship set *Teaches*, this means that:
 - ▶ A given entity $p_1 \in Professor$ can appear in any number of relationships $(p_1, c) \in Teaches$ where $c \in C$
 - ▶ A given entity $c_1 \in Course$ appears in at least one relationship $(p, c_1) \in Teaches$ where $p \in Professor$
- ▶ Observe that, typically, just the values 0, 1, n are used. We do not specify a number, e.g., 4, with this notation

Relationship sets

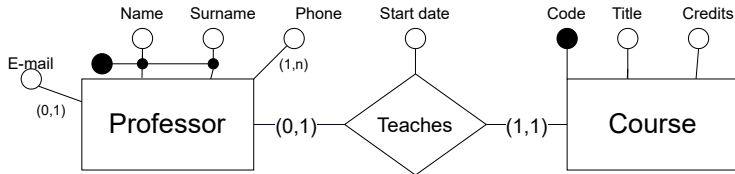
Attributes



- ▶ Relationship sets can also have attributes
- ▶ This is quite natural when the attribute tells us something *regarding the specific entities involved* in the relationship
- ▶ For instance, here we track the date in which a given professor started teaching a given course
- ▶ Here, can we keep track of evidences of a professor teaching the same course in different years?

Relationship sets

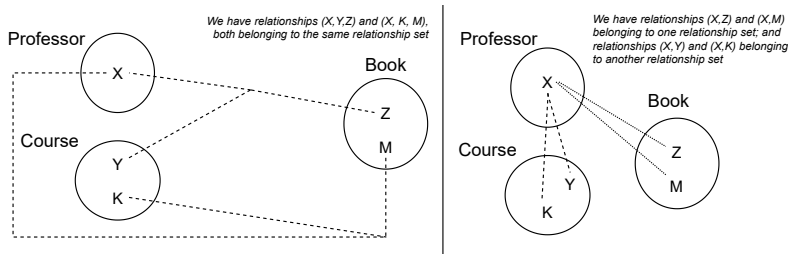
Attributes



- ▶ Relationship sets can also have attributes
- ▶ This is quite natural when the attribute tells us something *regarding the specific entities involved* in the relationship
- ▶ For instance, here we track the date in which a given professor started teaching a given course
- ▶ Here, can we keep track of evidences of a professor teaching the same course in different years?
 - ▶ **No.** We are going to see the *reification* process.

Relationships involving > 2 entities

- ▶ Sometimes, it is necessary to involve more than 2 entities in a relationship (though in a DB most of them are binary, and nearly always at most ternary)
- ▶ For instance:
 - ▶ *professor X teaches course Y using the book Z*
 - ▶ *professor X teaches course K using the book M*



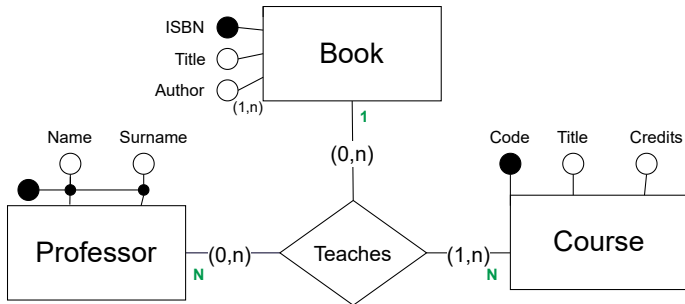
- ▶ Observe how the relationships on the right convey less information than those on the left. Can we have something equivalent using 3 relationship sets?

Relationship sets

Notation for ternary relationship sets

Relationship sets

Notation for ternary relationship sets



- ▶ Participation/cardinality constraints still apply to ternary relationships, although things complicate a little bit
- ▶ Here: a professor may teach zero or more courses, each course is taught by at least one professor, and a book is used within zero or more teaching activities, although a professor uses only a book within the same course

Relationship sets

Ternary relationship: setting participations/cardinalities

- ▶ If I have an entity of *Professor*, how many entities of *Book* and *Course* together can be associated to it?
 - ▶ Here, the answer is zero or more, so entity set *Professor* participates with $(0, n)$ to *Teaches*
- ▶ As for the green numerosities, they allow us to solve some ambiguity. Without them, we wouldn't know if a professor may appear in more than one relationship because:
 - ▶ He may teach more than one course using the same book
 - ▶ He may teach only one course but using multiple books
 - ▶ He may teach multiple courses using multiple books
- ▶ Setting the green 1 close to *Book*, we say that, given a *Professor* and a *Course*, there can be only a *Book* associated to them both. On the contrary, given a *Professor* and a *Book*, we may have more courses (green N close to *Course*)

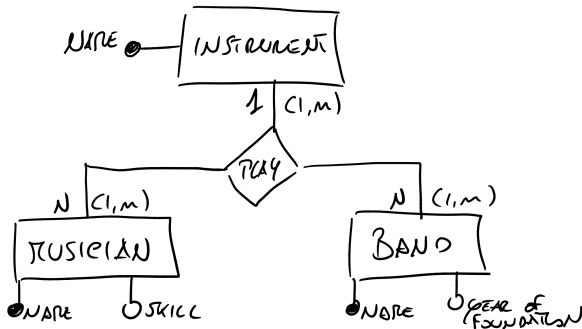
Relationship sets

Another ternary relationship example

- ▶ Let us consider the following scenario about bands, musicians, and kinds of instruments, that should be modeled using a ternary relationship
- ▶ A musician is identified by his/her name, and characterized by his/her skill level
- ▶ A band is identified by its name, and characterized by a genre and a year of foundation
- ▶ A kind of instrument is identified by its name (e.g., violin, piano, tenor saxophone, etc)
- ▶ Each musician can play in one or more bands, and a band has, in general, one or more musicians
- ▶ A musician can only play a single kind of instrument in a given band; however, in a band, there can be more than one musician playing the same kind of instrument

Relationship sets

Another ternary relationship example



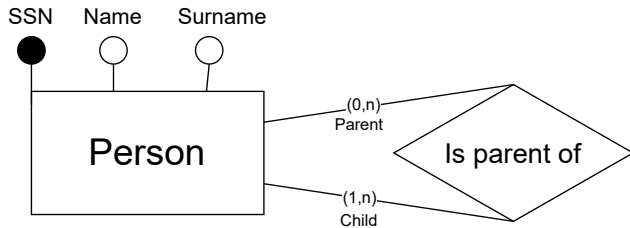
Note: if there is no necessity of modeling *Instrument* as an entity on its own, it is enough to employ just a many-to-many binary relationship *Play* between *Musician* and *Band*, with an attribute *Instrument* (cannot do that if the 1 below *Instrument* is replaced with an *N*, no multi-valued attributes on relationships)

Relationship sets

Recursive Relationship and Roles

Relationship sets

Recursive Relationship and Roles



- ▶ Is it possible to have a relationship between an entity set and itself (recursive, $R \subseteq E_1 \times E_1$)
 - ▶ For instance, consider the case in which we want to track the prerequisites of a given course
- ▶ In case of non-symmetric relationship, each occurrence of an entity set plays a “role” in it
 - ▶ In that case, it is useful to write down the **roles** as labels on the arcs

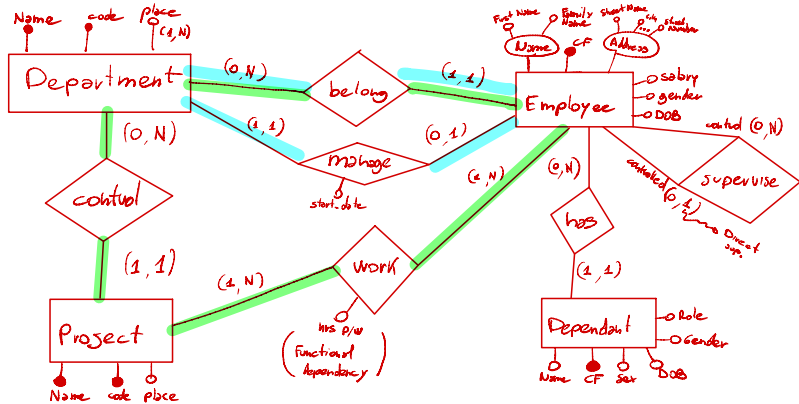
Example

Let us summarise an ER diagram that records information on the employees, the departments and projects of a company, starting from the following requirements:

- ▶ The company is organised into departments. Each department has a name, a unique number and a specific employee who manages it (manager). We want to keep track of the date on which that employee assumed that role. Each department may have several locations.
- ▶ A department controls a certain set of projects, each of which is marked by a single name, a single number and a single location.
- ▶ Each employee is identified by a name, a fiscal code, which uniquely identifies him/her, an address, a salary, a gender and a date of birth. Each employee belongs to a single department, but may work on several projects, not necessarily controlled by the same department. We want to keep track of the number of hours per week that an employee devotes to each project. Finally, we want to keep track of the direct supervisor of each employee.
- ▶ For insurance purposes, let's keep track of each employee's dependents, precisely their name, fiscal code, sex, date of birth and gender, and relationship to the employee (child, spouse, parent, etc.).

Example

Solution



Cycles:

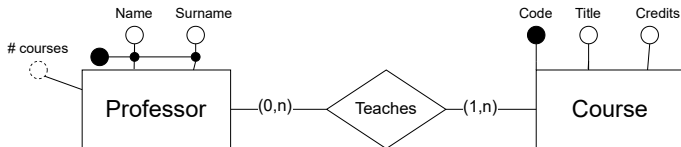
An employee can manage a dept. to which it does not belong to → **ISSUE!!!**

→ New Integrity constraint: An employee cannot ...

An employee can belong to a dept. while working on projects controlled by different depts.
→ OK by specifications

Derived attributes

- ▶ A derived attribute is an attribute whose value can be computed from other information
 - ▶ For instance, the number of courses taught by a professor can be derived from the number of relationships of relationship set *Teaches* he/she participates to
- ▶ Derived attributes are never manually inputted/updated by a DB user, and they require the set-up of proper update routines to keep their values consistent
- ▶ The derived attribute is depicted with a dashed circle
- ▶ The E-R diagram should include notes, written in natural language, on how to calculate the derived attributes contained in it
 - ▶ “*#courses of an entity of Professor can be calculated retrieving the number of relations Teaches in which such an entity participates*”



Weak entity sets

- ▶ Consider the following scenario:
 - ▶ We want to keep track of hotels, each characterized by a name
 - ▶ There may be multiple hotels with the same name in different cities, however, given a city, there cannot be two hotels with the same name
 - ▶ Each city is univocally identified by a code
- ▶ Intuitively, to univocally identify a hotel, its name is not enough; we also need to know the city where it is
- ▶ Similar scenario, about a cinema franchise domain: cinema hall number within a given multiplex cinema

Weak entity sets

(cont.)

- ▶ The notion of **weak entity set** formalizes the above intuition. A weak entity set is one whose existence of each of its entities is dependent on an entity of another entity set, called its **identifying entity (set)**; instead of associating a key with a weak entity, to uniquely identify it we use the identifying entity, along with extra attributes called **partial keys**
- ▶ Each weak entity must be associated with a *corresponding* identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set. The identifying entity set is said to **own** the weak entity set that it identifies. The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**

Weak entity sets

(cont.)

Note that:

- ▶ The weak entity set has a total participation in the identifying relationship with a cardinality of 1 (i.e., (1,1))
- ▶ The identifying entity set can have a partial participation in the identifying relationship, and its cardinality is typically N
- ▶ Attributes involved in the (partial) keys must have (1, 1) cardinality
- ▶ Chains of weak entity sets, dependence over more than one identifying entity set, and multiple sets of partial keys are allowed; yet, cycles must be avoided

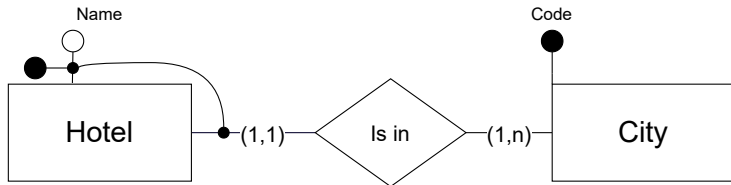
Weak entity sets

Notation

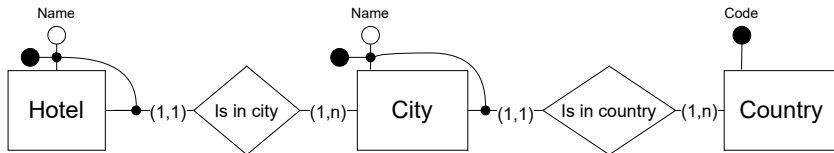
Weak entity sets

Notation

- Observe the $(1,1)$ cardinality on the *Hotel* side



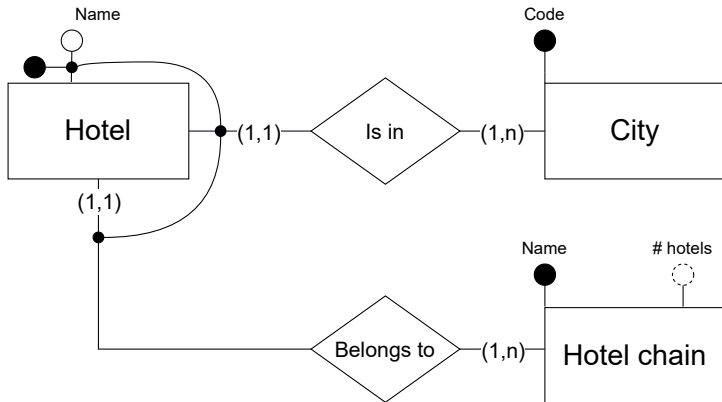
- Example of a chain of weak entity sets



Weak entity sets

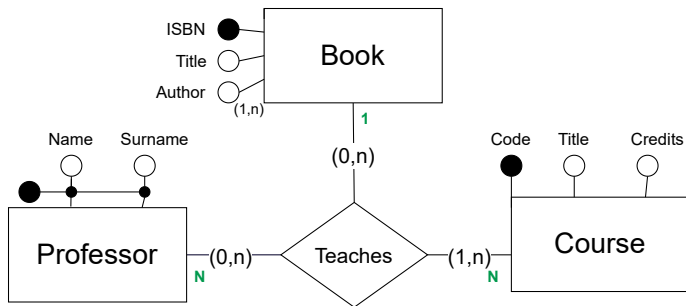
Notes

- ▶ A weak entity set may depend on more than one identifying entity



Ternary relationship sets

Question



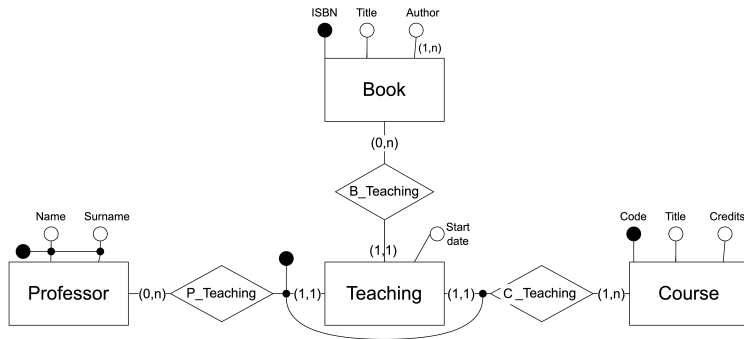
Can we remove the ternary relationship by doing something more than what we saw before, i.e., trying to use 3 relationship sets?

Ternary relationship set reification

- ▶ Through a *reification* process we can replace a ternary relationship set with an entity set and 3 relationship sets

Ternary relationship set reification

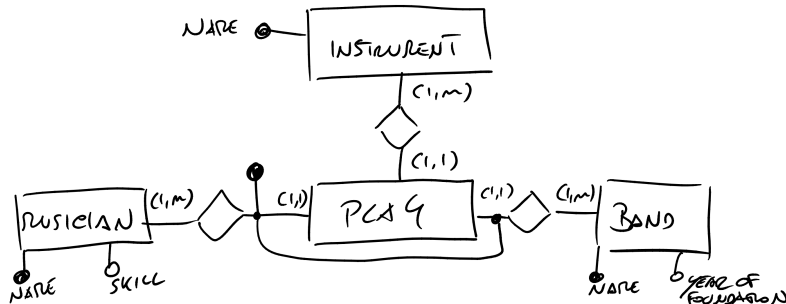
- Through a *reification* process we can replace a ternary relationship set with an entity set and 3 relationship sets



- The obtained representation is equivalent
- Observe that the introduced entity set is weak with respect to two entity sets: what if the professor can use more than one book in a course?

Ternary relationship sets

Another reification example

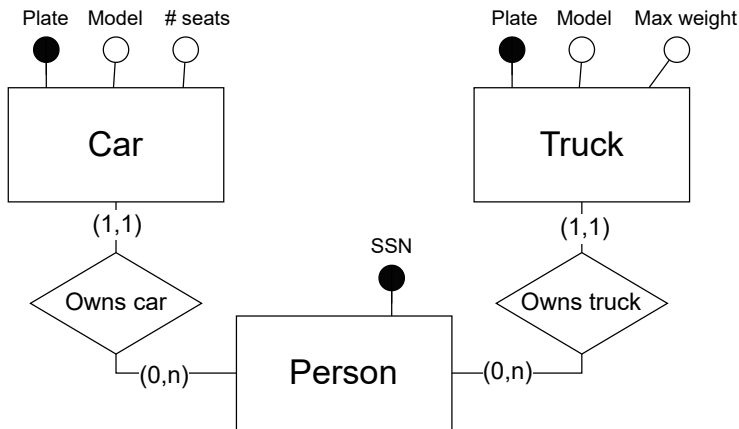


What if...

- ▶ *Play* is weak w.r.t. *Instrument*, *Musician*, *Band*?
- ▶ *Play* is weak w.r.t. *Instrument* and *Musician*?
- ▶ *Play* is weak w.r.t. *Instrument*, *Band*?

Specialization

- ▶ It may happen that different entity sets share some commonalities regarding their attributes or relationships
 - ▶ E.g., a *Truck* and a *Car* may both be considered as a *Vehicle*



Specialization

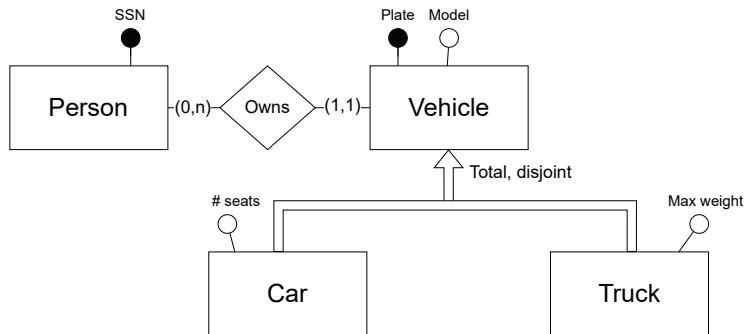
Notation

- ▶ The **specialization** construct allows us to handle such a scenario
- ▶ A “child” entity set inherits all the attributes and relationship participations of the “parent” entity set which it is linked to
- ▶ Each occurrence of a child entity is also an occurrence of the parent entity (but not vice versa!!!)

Specialization

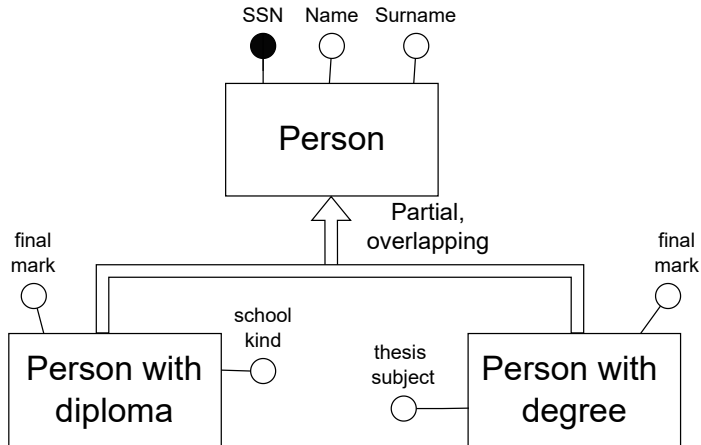
Notation

- ▶ The **specialization** construct allows us to handle such a scenario
- ▶ A “child” entity set inherits all the attributes and relationship participations of the “parent” entity set which it is linked to
- ▶ Each occurrence of a child entity is also an occurrence of the parent entity (but not vice versa!!!)



Specialization

Another example



Specialization

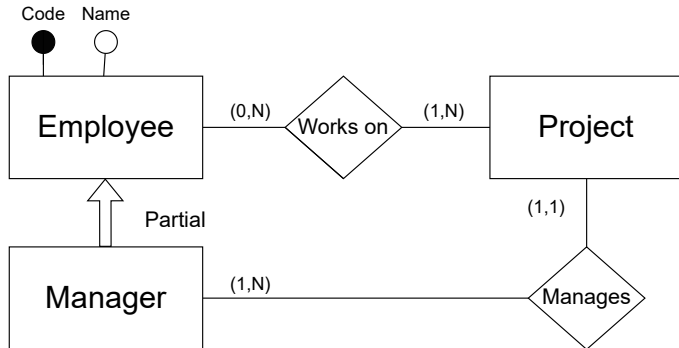
Two orthogonal dimensions

- ▶ A specialization may be:
 - ▶ **Total** or **Partial**
 - ▶ **Total**: each higher-level entity corresponds to at least one of the lower-level entities (e.g., a *Vehicle* is a *Car* or a *Truck*)
 - ▶ **Partial**: there may be higher-level entities that do not correspond to any lower-level entity (e.g., a *Person* may not hold a diploma nor a degree)
 - ▶ **Disjoint** or **Overlapping**
 - ▶ **Disjoint**: a higher-level entity corresponds to at most one of the lower-level entities (e.g., a *Vehicle* can be a *Car* or a *Truck*, but not both)
 - ▶ **Overlapping**: a higher-level entity may correspond to more lower-level entities (e.g., a *Person* may hold both a diploma as well as a degree)
- ▶ Of course, each lower-level entity corresponds to a higher-level entity
- ▶ **Always write** in the diagram the kind of specialization

Specialization

(cont.)

A specialization can also have a single child



Note how the (natural) domain constraint “a manager can only manage projects he works on” is not enforced by the schema

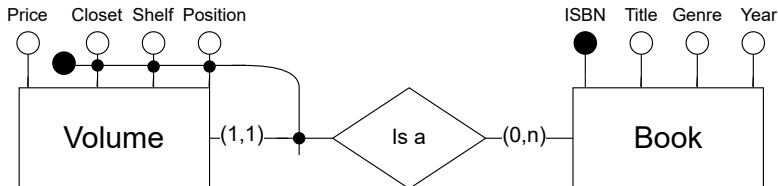
Design patterns

- ▶ We have finished discussing all the elements of our E-R diagrams
- ▶ We will now introduce the so-called design patterns, i.e., “conventional ways” to represent scenarios one may encounter while modeling a domain, as proposed by the authors of the book we consider for the course
- ▶ Each design pattern is associated with a name
 - ▶ *Caveat 1*: it's just a name, useful for mnemonic purposes... it does not come with any profound philosophical meanings
 - ▶ *Caveat 2*: one could envision slight variations that could occur within a given design pattern

Design patterns

The *instance of construct*

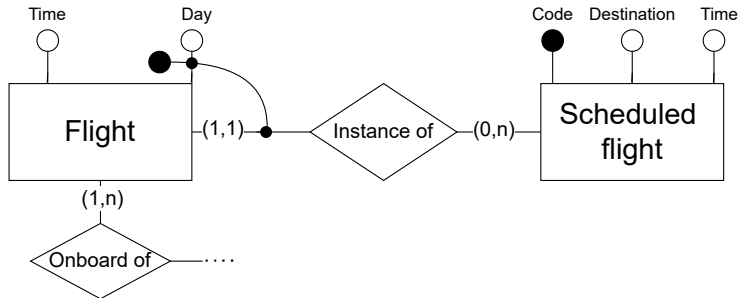
- ▶ Consider the following scenario:
 - ▶ A library maintains a catalogue of published books. For each book, it records: ISBN, title, genre, and year of publication
 - ▶ Not all books are available at the library. Of the available ones, we record the selling price and the location of each copy: closet, shelf, position on the shelf
- ▶ We may notice the presence of two different entities:
 - ▶ *Book*: a “virtual” book, with its ISBN
 - ▶ *Volume*: the actual, “physical” copy, which can be considered as an instance of a book



Design patterns

The *instance of* construct

- ▶ An airport wants to keep track of flights
- ▶ A flight is uniquely identified by a code, and characterized by a destination and a daily scheduled departure time (e.g., the AZ123 flight of 3:50 PM to Rome)
- ▶ For each occurrence of a flight, we also want to keep track of the actual departure time, its crew, [...]



Design patterns

The *instance of* and *is part of* constructs

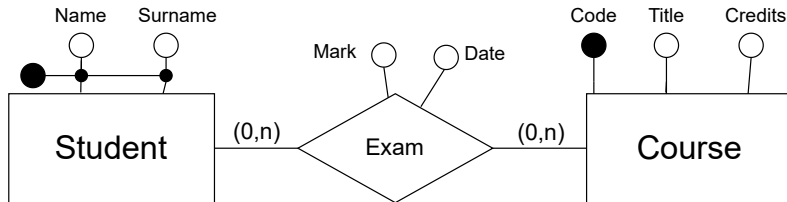
Note that:

- ▶ Typically the *instance of* construct is used for one to many relationships
- ▶ The construct *is part of* behave similarly, representing a composition relationship. Here, we shall not use a weak entity if the entity has an autonomous existence:
 - ▶ A *cinema hall* entity is part of a cinema entity (cinema hall is a weak entity set, as it can not exist without a cinema)
 - ▶ A technician entity is part of a team entity (the technician exists beyond its team, thus, it is a proper entity)

Design patterns

Multiple relationships between entities

The following solution is valid if and only if a given student can only take the exam for a course *once*

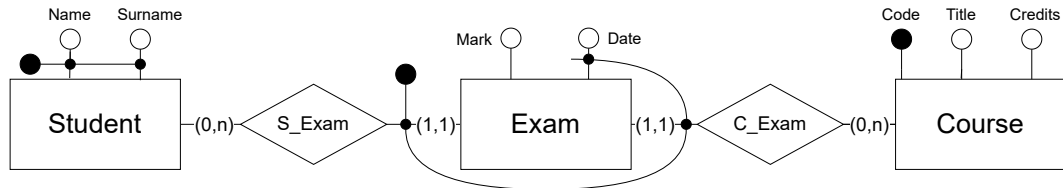


This is because the relationship set *Exam* is composed of unique pairs *Student–Course* (the fact that the relationships have some attributes does not change that)

Design patterns

Multiple relationships between entities

To allow a student to take more than once the exam for a course, we must employ a reification process



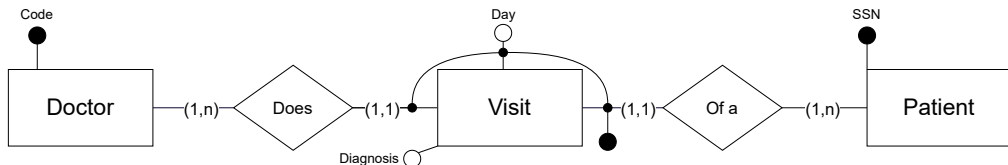
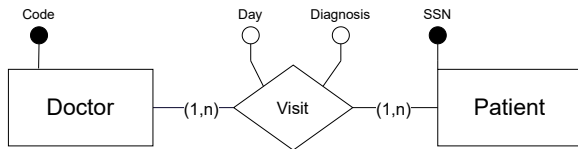
Note that

- ▶ the attribute *Date* belongs now to the key of the entity *Exam*
- ▶ An alternative solution might be associating a *Code* attribute to *exam* acting as the key, and relying on a strong entity instead of a weak one.

Design patterns

Multiple relationships between entities

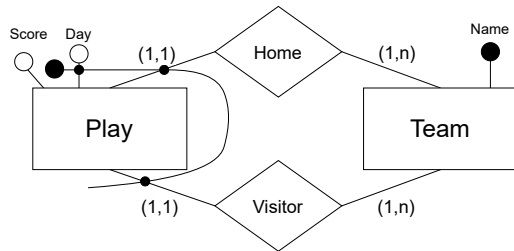
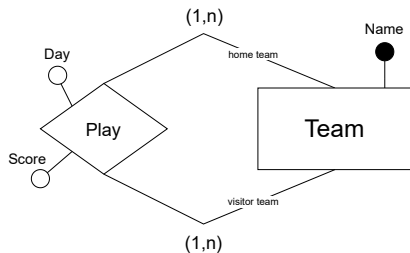
Assume that we want to model, within a hospital domain, information about doctors, patients, and visits. . . can you spot any differences between the two solutions?



Design patterns

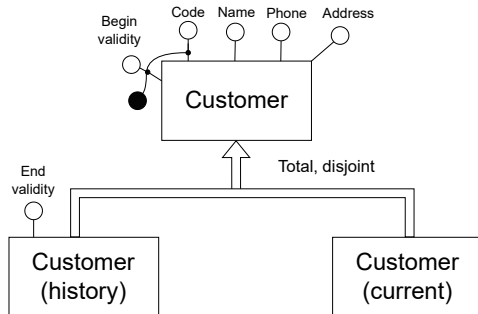
Multiple relationships between entities

Finally, observe that the same strategy might apply to recursive relationships



Design patterns

Managing history – Entities

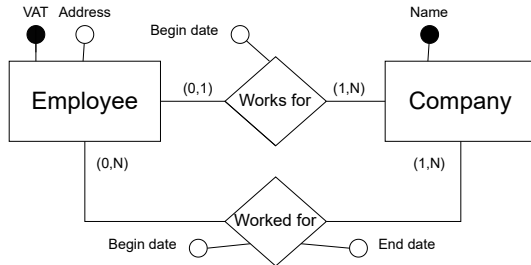


Domain constraints:

- ▶ $Begin\ validity < End\ validity$
- ▶ A given *Customer* must appear exactly once as a *Customer (current)*
- ▶ A given *Customer* may appear multiple times as a *Customer (history)*

Design patterns

Managing history – Relationships



Domain constraints:

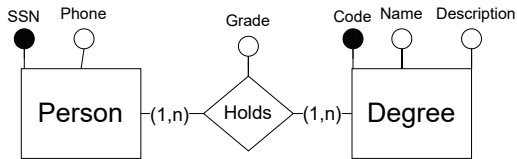
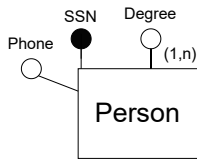
- ▶ A given *Employee* must participate to at least one between *Works for* or *Worked for*, possibly both

Question: How can we represent that a person might have worked, at different times, for the same company?

Design patterns

Attributes vs entity/relationship sets

- ▶ When should attributes be used, and when entity and relationship sets instead?
- ▶ It really depends on the domain, and on the amount of detail that we want to express



Documentation of E-R diagrams

- ▶ While being highly expressive, an E-R diagram is seldom enough to encode all the requirements of a given application
- ▶ For instance, it does not convey the semantics behind entity set names
 - ▶ Does an entity set *Project* refer to something internal or external to the company?
- ▶ In addition, some domain constraints cannot simply be expressed using the E-R notation
 - ▶ The salary of an employee must be lower than that of his/her supervisor
 - ▶ An employee can be manager only of the department to which he/she belongs
- ▶ Thus, E-R diagrams are typically complemented by natural language notes that further describe their semantics and all the constraints that were not captured in them

Documentation of E-R diagrams

Business rules

Business rules are often used to specify domain properties not representable in an E-R diagram. They can be:

- ▶ **Concept description** - precise definition of an entity, relationship, or attribute, relevant to the application
- ▶ **Integrity constraint** - cardinality constraints or other types of constraints not definable by means of E-R diagrams
- ▶ **Derivable concepts** - concepts that can be obtained by other inference or arithmetical operations over other concepts already presented in the schema

Documentation of E-R diagrams

Business rules - assertions and derivations

Rules that describe integrity constraints can be expressed by means of *assertions*:

- ▶ they are statements that must be always verified in the DB
- ▶ they are atomic, thus cannot be decomposed in sentences that are assertions too
- ▶ they should be written in a declarative way, i.e., not suggesting a way to satisfy them
- ▶ typical form: *concept must / must not expression over the concepts*
 - ▶ the manager of a department must belong to such a department
 - ▶ an employee must not have a salary greater than the manager of the department to which he/she belongs

Derivation rules describe the operations to derive a concept:

- ▶ typical form: *concept is obtained by operations over the concepts*
 - ▶ the number of employees working on a department is obtained by counting the employees that belong to such a department

Documentation of E-R diagrams

Business rules - enforcement

Assertions and *derivation* rules must be encoded at logical or physical level, so as to actually enforce them in the DB:

- ▶ define specific constraints at the logical level (e.g., it with `CHECK CONSTRAINT` instructions)
- ▶ use active rules, e.g., those that reacts to some chosen instructions and perform operations on the DB consequently (triggers)
- ▶ define SQL functions to constraint/guide the interaction with the DB, especially for insertion/update/delete operations over data
- ▶ devise procedure by means of external, programming languages (this is the worst solution)

Exercise 1, TV series

Let us synthesize the ER conceptual schema for a database of TV series data recording information about episodes, actors and directors, based on the following requirements:

- ▶ Let us assume that both actors and directors are uniquely identified by their name and surname, and are characterized by their birth date and their nationality. The same person may be both an actor as well as a director
- ▶ Each TV series is characterized by its title, genre, release year, and director. We assume that there is only one director per series

Exercise 1, TV series

(cont.)

- ▶ It may be the case that distinct TV series have the same title (this is the case, for instance, with remakes). Nevertheless, given a year, there cannot be two series with the same title
- ▶ A TV series is composed of a number of episodes, each with an incremental number that uniquely identifies it within the series, and a title
- ▶ Different episodes may involve different actors, each with a specific role. For the sake of simplicity, let us assume that, in a given episode, each actor may play one role only

Exercise 2, University DB

Let us synthesize the ER conceptual schema of a database for the management of a university on the basis of the following set of requirements.

- ▶ Each course offered by the university is uniquely identified by its numerical code and characterized by a name, a set of prerequisites (a set of other courses that must be already passed before attending the course), a number of hours, and the professors that are allowed to teach it
- ▶ Professors are partitioned into full and associate professors. Each professor is uniquely identified by his/her fiscal code and characterized by a first and a last name, one or more mobile numbers, possibly an email address, and the department to which he/she belongs. For full professors, we store the day they were promoted

Exercise 2, University DB

(cont.)

- ▶ Each university's department is uniquely identified by its name and characterized by a code and a budget. Every department has a manager, selected among the full professors. In addition, we want to keep track of the number of professors belonging to each department
- ▶ A given course can be offered in different years. We call each course offering an “edition” of a course (e.g., the *databases* course of the year 2022). Each course edition is taught by one or more professors, among those allowed to teach it
- ▶ For teaching purposes, a given professor can use one or more books for each course edition in which he/she is involved. A book is identified by its ISBN, and has a title, editor, edition, and one or more authors

Exercise 3

Design an ER schema for a database storing information regarding people who are resident or buried in a given city:

- ▶ For each person we are interested in storing the name, surname, tax code (uniquely identifying the person), gender, and date of birth. A person can be resident (living) or buried (deceased) in the city. For each living person we want to store the address (street, house number), one or more telephone numbers, and marital status
- ▶ Let us assume that the city has several cemeteries, each identified by an alphanumeric code and characterized by a name and an address. Within each cemetery, each burial site is identified by a numeric code (thus, burial sites located in different cemeteries may have the same number)
- ▶ For each deceased person buried in a cemetery of the city, we store the burial site (note that some burial sites are collective, e.g., family sites), date of death, age at the time of death, and death cause

Exercise 4

Let the following set of requirements for a database for the management of a library be given.

- ▶ The object of library loans are copies of individual volumes, uniquely identified by an inventory number. Each volume is uniquely identified by the ISBN code and is related to a specific edition of a work. Each edition may consist of one or more volumes. The number of volumes may vary from edition edition to edition. For example, the text on databases by Ceri and colleagues in some editions consists of a single volume, in others of two volumes.
- ▶ Each volume may exist in one or more copies.
- ▶ Each edition is characterised by the work, the series and the year of publication. Let us assume that different editions of the same work, within the same series, cannot be published in the same year
- ▶ Each work has a title, one or more authors and a year of first publication. Let us assume for the sake of simplicity that each work is uniquely identified by its title.

Exercise 4

(cont'd)

- ▶ Each series has a name and a publisher. Each publisher has a name, which uniquely identifies it. The name of the series is unique among the series published by a given publisher (there cannot be two series with the same name published by a given publisher, but we cannot exclude that different publishers give the same name to a series).
- ▶ For each current loan (those already completed are of no interest), the user who made the loan (uniquely identified by a code and characterised by first name, surname and telephone number) and the expected return date are recorded. Let us assume that a user can borrow several copies (of different volumes) at the same time.

Define an Entity-Relationships diagram describing the information content of the system, clearly illustrating any assumptions made. The schema must be completed with reasonable attributes for each entity (identifying possible keys) and relationship. The cardinality and participation constraints of each relationship must be carefully specified. Any business rules (derivation rules and integrity constraints) necessary to encode some of the expected requirements of the system should also be defined.

Exercise 5

Let the following set of requirements for a database for managing a set of workshops be given.

- ▶ Each workshop is uniquely identified by name and is characterised by an address, a telephone number an email address, a manager and the number of employees. Assume that each workshop has one and only one manager and at least one employee and that one manager leads one and only one workshop.
- ▶ Employees and managers are uniquely identified by their tax code and are characterised by their first name, surname, home address, one or more telephone numbers and the number of years of service at the workshop. Also keep track of the age of the managers. Assume that a manager may be an employee of the workshop he/she leads, but is not necessarily an employee of the workshop (i.e. in some cases, the manager may not be an employee of the workshop he/she leads).
- ▶ Each repair is carried out by one and only one workshop and concerns one and only one vehicle. Of each repair is recorded, the code (unique within the workshop), the time and date of acceptance of the vehicle and, in the case of a completed repair, the time and date of vehicle return.

Exercise 5

- ▶ The vehicle model, type, number plate, year of registration and owner are recorded. Each vehicle has one and only one owner. Vehicle owners are registered with their tax code, first name and surname, residential address and one or more of phone numbers

Define an Entity-Relationships diagram describing the information content of the system, clearly illustrating any assumptions made. The schema must be completed with reasonable attributes for each entity (identifying possible keys) and relationship. The cardinality and participation constraints of each relationship must be carefully specified. Any business rules (derivation rules and integrity constraints) necessary to encode some of the expected requirements of the system should also be defined.